Latent Variable Models

General Principles

In some scenarios, the observed data does not directly reflect the underlying structure or factors influencing the outcome. Instead, *latent variables*—variables that are not directly observed but are inferred from the data—can help model this hidden structure. These latent variables capture unobserved factors that affect the relationship between predictors (X) and the outcome (Y).

We model the relationship between the predictor variables (X) and the outcome variable (Y) with a latent variable (Z) as follows:

$$Y = f(X, Z) + \epsilon$$

Where: - Y is the observed outcome variable. - X is the observed predictor variable(s). - Z is the latent (unobserved) variable, which we aim to infer. - *f(X, Z)* is the function that relates X and Z to Y. - is the error term, typically assumed to be normally distributed with mean 0 and variance $\hat{}2$.

The latent variable Z can represent various phenomena, such as group-level effects, time-varying trends, or individual-level factors, that are not captured by the observed predictors alone.

Considerations

In Bayesian regression with latent variables, we consider the uncertainty in both the observed and latent variables. We declare prior distributions for the latent variables, in addition to the usual priors for regression coefficients and intercepts. These latent variables are often modeled using Gaussian distributions (*Normal* priors) or more flexible distributions such as *Multivariate Normal* for correlations among the latent variables.

The goal is to infer the posterior distribution over both the parameters and the latent variables, given the observed data.

Example

Below is an example code snippet demonstrating Bayesian regression with latent variables using TensorFlow Probability:

Python

```python
from BayesForge import bf
import jax.numpy as jnp


# Setup device-------------------------------------------------
m = bf(platform='cpu')

# Data Simulation ----------------------------------------------
NY = 4  # Number of dependent variables or outcomes (e.g., dimensions for latent variables)
NV = 8  # Number of observations or individual-level data points (e.g., subjects)
N = 100
K = 5
a = 0.5
# Generate the means and offsets for the data
# means: Generate random normal means for each of the NY outcomes
# offsets: Generate random normal offsets for each of the NV observations
means = m.dist.normal(0, 1, shape=(NY,), sample=True, seed=10)
offsets = m.dist.normal(0, 1, shape=(NV, 1), sample=True, seed=20)
# observation noise (sd 0.5)
noise = m.dist.normal(0, 0.5, shape=(NV, NY), sample=True, seed=30)

Y2 = offsets + means + noise

# Simulate individual-level random effects (e.g., random slopes or intercepts)
# b_individual: A matrix of size (N, K) where N is the number of individuals and K is the nu
b_individual = m.dist.normal(0, 1, shape=(N, K), sample=True, seed=0)

# mu: Add an additional effect 'a' to the individual-level random effects 'b_individual'
# 'a' could represent a population-level effect or a baseline
mu = b_individual + a

# Convert Y2 to a JAX array for further computation in a JAX-based framework
Y2 = jnp.array(Y2)
```

```python
# Set data ------------------------------------------------
dat = dict(
    NY = NY,
    NV = NV,
    Y2 = Y2
)
m.data_on_model = dat

# Define model --------------------------------------------------
def model(NY, NV, Y2):
    means = m.dist.normal(0, 1, shape=(NY,), name='means')
    offset = m.dist.normal(0, 1, shape=(NV, 1), name='offset')
    sigma = m.dist.exponential(1, shape=(NY,), name='sigma')
    tmp = jnp.tile(means, (NV, 1)).reshape(NV, NY)
    mu_l = tmp + offset
    m.dist.normal(mu_l, jnp.tile(sigma, [NV, 1]), obs=Y2)

# Run sampler ---------------------------------------------------
m.fit(model, progress_bar=False)

# Summary -------------------------------------------------------
m.summary()
```

```
An NVIDIA GPU may be present on this machine, but a CUDA-enabled jaxlib is not installed. Fal

bf v 0.0.60 package loaded
jax.local_device_count 16
```

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_sd
means[0]	-1.17	0.36	-1.73	-0.59	644.50	978.46	1.01	0.01	0.01
means[1]	-0.51	0.29	-0.97	-0.04	527.82	576.15	1.01	0.01	0.01
means[2]	1.40	0.30	0.92	1.88	543.58	802.98	1.01	0.01	0.01
means[3]	-0.51	0.32	-1.00	0.02	517.70	803.64	1.01	0.01	0.01
offset[0, 0]	-0.64	0.32	-1.14	-0.13	519.28	674.08	1.01	0.01	0.01
offset[1, 0]	0.80	0.31	0.31	1.31	595.72	727.20	1.01	0.01	0.01
offset[2, 0]	-0.83	0.33	-1.38	-0.29	574.57	731.88	1.01	0.01	0.01
offset[3, 0]	0.32	0.34	-0.22	0.88	375.33	517.19	1.01	0.02	0.01
offset[4, 0]	-0.64	0.33	-1.14	-0.10	415.31	804.57	1.01	0.02	0.01
offset[5, 0]	-0.31	0.34	-0.87	0.20	512.55	682.10	1.01	0.01	0.01

	mean	sd	eti89_lb	eti89_ub	ess_bulk	ess_tail	r_hat	mcse_mean	mcse_sd
offset[6, 0]	0.07	0.32	-0.43	0.60	561.47	826.86	1.01	0.01	0.01
offset[7, 0]	0.44	0.32	-0.06	0.97	574.77	798.42	1.01	0.01	0.01
sigma[0]	0.68	0.24	0.40	1.11	1368.88	1592.84	1.00	0.01	0.01
sigma[1]	0.27	0.16	0.08	0.54	229.84	59.86	1.03	0.01	0.00
sigma[2]	0.37	0.17	0.14	0.65	582.68	600.04	1.01	0.01	0.01
sigma[3]	0.46	0.17	0.25	0.76	1269.92	1365.43	1.01	0.00	0.01

R

```r
library(BayesForge)
jnp <- reticulate::import("jax.numpy")

# Setup device ------------------------------------------------
m <- importBF(platform = "cpu")

# Data Simulation ---------------------------------------------
NY <- 4L  # Number of dependent variables or outcomes (e.g., dimensions for latent variables)
NV <- 8L  # Number of observations or individual-level data points (e.g., subjects)
N <- 100L
K <- 5L
a <- 0.5
means <- m$dist$normal(0, 1, shape = reticulate::tuple(NY), sample = TRUE, seed = 10L)
offsets <- m$dist$normal(0, 1, shape = reticulate::tuple(NV, 1L), sample = TRUE, seed = 20L)
noise <- m$dist$normal(0, 0.5, shape = reticulate::tuple(NV, NY), sample = TRUE, seed = 30L)

Y2 <- offsets + means + noise

b_individual <- m$dist$normal(0, 1, shape = reticulate::tuple(N, K), sample = TRUE, seed = 0L
mu <- b_individual + a

Y2 <- jnp$array(Y2)

# Set data ----------------------------------------------------
m$data_on_model <- list(NY = NY, NV = NV, Y2 = Y2)

# Define model ------------------------------------------------
model <- function(NY, NV, Y2) {
  means <- m$dist$normal(0, 1, shape = reticulate::tuple(NY), name = "means")
  offset <- m$dist$normal(0, 1, shape = reticulate::tuple(NV, 1L), name = "offset")
```

```r
  sigma <- m$dist$exponential(1, shape = reticulate::tuple(NY), name = "sigma")
  tmp <- jnp$reshape(jnp$tile(means, reticulate::tuple(NV, 1L)), reticulate::tuple(NV, NY))
  mu_l <- tmp + offset
  m$dist$normal(mu_l, jnp$tile(sigma, list(NV, 1L)), obs = Y2, name = "y")
}

# Run sampler ------------------------------------------------
m$fit(model, progress_bar = FALSE)

# Summary ----------------------------------------------------
m$summary()
```

Julia

```julia
using PyBayesForge
using PythonCall

# Setup device -----------------------------------------------
m = importBF(platform="cpu")

# Data Simulation --------------------------------------------
NY = 4  # Number of dependent variables or outcomes (e.g., dimensions for latent variables)
NV = 8  # Number of observations or individual-level data points (e.g., subjects)
N = 100
K = 5
a = 0.5
means = m.dist.normal(0, 1, shape=(NY,), sample=true, seed=10)
offsets = m.dist.normal(0, 1, shape=(NV, 1), sample=true, seed=20)
noise = m.dist.normal(0, 0.5, shape=(NV, NY), sample=true, seed=30)

Y2 = offsets + means + noise

b_individual = m.dist.normal(0, 1, shape=(N, K), sample=true, seed=0)
mu = b_individual + a

Y2 = jnp.array(Y2)

# Set data ---------------------------------------------------
m.data_on_model = pydict(NY = NY, NV = NV, Y2 = Y2)

# Define model -----------------------------------------------
```

```
@BF function model(NY, NV, Y2)
    means = m.dist.normal(0, 1, shape=(NY,), name="means")
    offset = m.dist.normal(0, 1, shape=(NV, 1), name="offset")
    sigma = m.dist.exponential(1, shape=(NY,), name="sigma")
    tmp = jnp.reshape(jnp.tile(means, (NV, 1)), (NV, NY))
    mu_l = tmp + offset
    m.dist.normal(mu_l, jnp.tile(sigma, (NV, 1)), obs=Y2, name="y")
end

# Run sampler ----------------------------------------------
m.fit(model, progress_bar=false)

# Summary --------------------------------------------------
m.summary()
```

Mathematical Details



Model validation tests

- Documentation examples: the Python and R examples of this page are run by the documentation tests (`Python/test_model_pages.py`, `R/test_r_model_pages.py`).